Ders 3 Genişletilmiş Notu: Nesne ve Sınıf, Bilgi Gizleme, UML Sınıf Diyagramı, Static, Final ve Yapıcı Metotlar

Ders 3 Genişletilmiş Notu: Nesne ve Sınıf, Bilgi Gizleme, UML Sınıf Diyagramı, Static, Final ve Yapıcı Metotlar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geliştirme Ortamı ve UML Aracı
- 2. Nesne, Üyeler ve Kapsülleme
- 3. Sınıf Tanımı ve Sınıftan Nesneye
- 4. UML Gösterimine Giriş ve Temel Terimler
- 5. Nesne Oluşturma ve Bellek
- 6. UML Ailesi ve Neden Kod Üreten Bir Araç
- 7. Her Nesne Ayrı Bir Bireydir
- 8. Nesneye Mesaj Gönderme
- 9. Paketler
- 10. Nesne Diyagramı ve Easy UML
- 11. Bilgi Gizleme
- 12. UML Sınıf Diyagramının Yazımı
- 13. Bilgi Gizlemenin Uygulanması
- 14. Static Alanlar
- 15. Static Metotlar
- 16. Final Alanlar ve Metotlar
- 17. Yapıcı Metotlar ve Varsayılan Değerler
- 18. Sonlandırıcı Metot ve Çöp Toplayıcı
- 19. Yapıcı Kuralları ve Örnekleme
- 20. Aşırı Yükleme
- 21. Nesneleri Karşılaştırmak: == ve equals
- 22. Akış Kontrolü ve main Metodu
- 23. Car Örneği: Kendi main Metodu Olan Sınıf
- 24. Car Örneğinin İkinci Sürümü ve Yanlış Tasarım
- 25. this Anahtar Kelimesi

1. Geliştirme Ortamı ve UML Aracı

Hoca, ana konuya geçmeden önce kullanılacak araçlara değindi. IntelliJ'i incelediğini, UML dışındaki işler için rahatça kullanılabileceğini, NetBeans'ten daha hafif göründüğünü söyledi. Ancak NetBeans'teki Easy UML kadar rahat bir UML çizme ve koddan diyagram çıkarma (reverse) imkânı olmadığını ekledi. Eclipse'i kendisi hiç kullanmadığını, yine de kullanılabileceğini belirtti. Bu yüzden UML tarafında NetBeans biraz öne çıkıyor.

2. Nesne, Üyeler ve Kapsülleme

2.1 İki Temel Kavram: Nesne ve Sınıf

Nesne yönelimli programlama iki temel kavrama dayanır: nesne (object) ve sınıf (class). Programlamanın ana öğesi nesnedir. Nesne somut (concrete) bir yapıdır, sınıf ise soyuttur (abstract). Nesne gerçek dünyadaki bir varlığa karşılık gelir.

Hoca derste "soyut" sözcüğünü kendi tanımı olarak kullandığını özellikle belirtti. Buradaki "soyut", ileride Java'da göreceğimiz `abstract` anahtar kelimesinden farklıdır.

2.2 Nesnenin İki Parçası

Bir nesne, temsil ettiği şeyin özelliklerini (attribute) ve görevlerini (task) bir arada tutar.

- **Özellik:** Nesnenin bir niteliğidir. Bir şeyin rengi, boyu ya da uzunluğu gibi.
- **Görev:** Nesnenin yaptığı bir eylemdir. Koşmak ya da kütüphanedeki bütün kitapları yazdırmak gibi.

Nesne, yapısal programlamada gördüğümüz değişkene benzer, ama ondan bir farkı vardır. Yapısal programlamada `struct` öğeleri yalnızca özellik tutar. Örneğin karmaşık sayı yapısında reel ve sanal kısım bulunur. Nesnelerde ise bunlara ek olarak metotlar da vardır.

Nesnenin özellikleri veridir ve bu veri iki biçimde tutulabilir:

- **Değer tipli (value type):** `int`, `float` gibi ilkel (primitive) tipler.
- **Referans tipli (reference type):** Sonradan tanımlanan yeni tipler. `Book` ya da `Author` gibi.

Nesnenin ikinci önemli parçası görevler, yani eylemler ya da metotlardır. Bunlara fonksiyon da denebilir, ama nesne yönelimli kavramlarda fonksiyondan farklı bir yanı vardır: Metot, nesnenin bütün özelliklerine erişebilir.

2.3 Metotlar Alanlara Nasıl Erişir?

Hoca farkı iki değişkenle gösterdi: `int x` ve `int y`. Yordamsal (procedural) programlamada fonksiyonlar bu değişkenlere ancak değişkenler global kapsamdaysa erişebilir. Nesne yönelimli programlamada ise değişkenler sınıf içinde tanımlıysa metotlar onlara doğrudan erişir. Aynı `x` ve `y` bir `f` fonksiyonunun içinde tanımlanırsa, başka bir metottan yine doğrudan erişilemez.

Kitap örneğinde `Book` bir sınıftır. `int id`, `float price` gibi özellikleri, `list` gibi bir metodu vardır. `list` metodu bu özelliklere doğrudan erişir. Bu erişimin `this` gibi anahtar kelimelerle ilgili kuralları da vardır. Hoca bunların sonraki konularda görüleceğini söyledi. Bu kuralları iyi öğrenmek amaçlardan biridir. Asıl ilk amaç ise canlı ya da cansız bir gerçek dünya varlığını modellemek, sonra da bu modelden somut nesneler oluşturmaktır.

Kısacası soyut kavram sınıf, somut olan nesne, üye (member) ise bunların özellikleri ve metotlarıdır. Hoca bu aşamada sınıf ve nesneyi aynı şemayla gösterdi. Ayrıntılara girildikçe ikisi hakkında farklı şeyler söyleneceğini belirtti.

2.4 Kapsülleme ve Bilgi Gizlemeye İlk Bakış

Kapsülleme (encapsulation), nesnenin verisi ile metotlarının birbirinden ayrılamaması demektir. `Book` sınıfının özellikleri ve metotları bir arada bulunur, işte kapsülleme budur.

Bilgi gizleme (information hiding), kapsüllemenin bir başka önemli yönüdür. Özelliklere ve metotlara metotlar üzerinden erişilir. Örneğin nesnenin özelliklerini listelemek için `list` metodu kullanılır.

Hoca bu kısmı Türkçe de özetledi. Sınıf genel ve soyut bir kavramdır, tek başına doğrudan bir anlam ifade etmez, bizim için bir modelleme aracıdır. Önümüze günlük hayattan bir senaryo gelir: Kitabın kimliği (ID) ve fiyatı vardır. Kitabın yazarı da vardır, ama yazar belki başka bir sınıfa aittir, çünkü yazarın yalnızca adı soyadı değil, başka birçok bilgisi olabilir. Yapılarda

olduğu gibi burada da referans tipli öğeler saklanır. Özellikler ve metotlar birbirinden ayrılmadan kapsüllenir ve böylece nesne yönelimli model kurulur.

Yapısal programlamadan farkımız iki noktada toplanır. Birincisi metot tanımlayabilmemiz, ikincisi sınıflar ya da nesneler arasındaki ilişkileri tanımlayabilmemizdir. Bu ilişkiler, veri tabanı kavramlarında olduğu gibi birebir (one to one) ya da bire çok (one to many) olabilir. İlişkiler derste ayrıca kullanılacak.

3. Sınıf Tanımı ve Sınıftan Nesneye

3.1 Sınıf Bir Şablondur

Slayttaki temel sınıf tanımı Java'daki biçimdedir. Benzer kavramlar C++ ve C#'ta da vardır. Sınıf, bir nesneyi tanımlayan şablondur (template). Nesne değişkenleri bu şablondan bazı anahtar kelimelerle tanımlanır. Program sınıf olarak kodlanır, `class` da derste görülen ilk önemli anahtar kelimedir. Ama asıl işi nesneler yapar.

Hocanın benzetmesi kurabiyedir: Sınıfı kurabiye kalıbı (cookie cutter), nesneleri de o kalıptan çıkan kurabiyeler olarak düşünebilirsiniz.

İkinci benzetme arabadır. Araba genel bir kavramdır ve otomobil, kamyon gibi birçok türü vardır. Bu kavramdan üretilmiş somut bir araç ise nesnedir.

Kod tarafında `class` anahtar kelimesinden sonra, değişken adlarındaki kurallara uygun bir ad verilir ve süslü parantezler arasına sınıfın tanımı yazılır. Bu yapıldığında sınıf oluşmuş olur, içi henüz boş olsa bile. Gerçek hayatta kullanılması için ise tanımlama yapmak gerekir. Hoca bunu araba satın almaya benzetti. Aynısı ilkel tipler için de geçerlidir: `int` diye bir kavram vardır, ama ondan değişken üretmedikçe bize bir faydası olmaz. Sınıf da böyle düşünülmelidir.

İlkel tip ile referans tip arasındaki fark, değişkenin neye eriştiğindedir. İlkel tipli değişken doğrudan değere erişir. Referans tipli değişken ise, işaretçide olduğu gibi, bellek konumunun referansına erişir.

3.2 Araba Örneği, Kimlik ve Eylemler

Belirli bir araba bir nesnedir. "Araba" değildir, çünkü araba sınıftır. Bu belirli arabanın model, plaka numarası ve renk gibi özellikleri vardır. Bunlar üye değişkenlerdir.

Bir nesnenin özelliklerinden biri genellikle onu tekil olarak tanımlayan mantıksal bir kimlik olarak seçilir. Arabanın plaka numarası, kişinin vatandaşlık numarası ya da öğrencinin öğrenci numarası bu işi görür.

Hoca özellikleri nesnenin "statik", eylemleri de "dinamik" öğeleri olarak adlandırdı. Eylemler metotlardır: Arabaya plaka numarasını sormak, arabayı satmak, sürmek, geri döndürmek gibi.

> **Ders sonrası not:** Hoca burada "statik" sözcüğünü "dinamik"in karşıtı olarak kullandı. Bu kullanım, büyük olasılıkla Bölüm 14'te anlatılan `static` anahtar kelimesiyle aynı anlama gelmiyor.

Sınıf ise nesnenin özelliklerini ve metotlarını tanımlayan koddur. Hocanın Türkçe anlatımıyla: "Bundan sonra araba diye bir şey tanımladım. Bu arabadan oluşacak bütün nesneler bu özelliklere ve bu metotlara sahip olacaktır." Sınıf böylece genel bir kavram olur. Bir programda istediğimiz sayıda sınıf ve bir sınıftan istediğimiz sayıda nesne tanımlayabiliriz. Otomobil genel sınıfsa, `A1`, `A2` gibi adlar bu sınıftan tanımlanan referanslardır.

3.3 Alanların İki Grubu

Nesnenin özellikleri iki önemli gruba ayrılır: ilkeller ve ilkel olmayanlar. Yapısal programlama dersinde çoğunlukla `int`, `float`, `char` gibi ilkel tipler kullanılmıştı. Bir nesnede bunların bileşimi bulunabilir, ama ilkel olmayan tipler de eklenebilir.

Kitap örneğinde `id` bir `int` olur. `price` ve `numberOfPages` da kitabın önemli üyeleridir. Yazar ise tasarım açısından ilginç bir durumdur. Yazarı `String` olarak tutmak mümkündür, ama nesne

yönelimli kavramlara uygun değildir. Sınıflarımızı nesne yönelimli kurallara göre tasarlamalıyız, bu yüzden yazar `Author` tipinde tanımlanır. Bütün özelliklerle metotların bileşimi `Book` sınıfını oluşturur.

4. UML Gösterimine Giriş ve Temel Terimler

4.1 Akış Şemasından UML'e

Sınıfları ve nesneleri görsel bir diyagramla göstermek için UML kullanılır. Hoca önce akış şemasını (flowchart) hatırlattı. Akış şemasıyla algoritmalar, matematiksel ifadeler, koşullar ve döngüler belirtiliyordu. Nesne yönelimli modellemede bu işi UML gösterimi görür.

Slaytta iki gösterim yan yanadır: sınıf diyagramı ve nesne diyagramı. Şimdilik `Car` boş bir sınıftır, içine bilgiler daha sonra eklenecektir. `kit` ise `Car` sınıfının bir nesnesidir.

```
+-----+        +-----------+
| Car |        | kit : Car |   (nesne kutusundaki yazı altı çizilidir)
+-----+        +-----------+
 sınıf            nesne
```

4.2 Önemli Nesne Yönelimli Terimler

Slayttaki terimler aynı kavramın farklı adlarıyla birlikte verilir:

- **Alan:** Veri (data), üye (member), alan (field), özellik (attribute), nitelik (feature, characteristic).
- **Durum (state):** Nesnenin bütün alanlarının o andaki değerlerinin kümesidir. Örneğin kitabın sayfa sayısı 0'dan n'e kadar bir değer alabilir.
- **Metot:** Görev (task), eylem (action), fonksiyon, metot, üye metot. Fonksiyona benzer.
- **Üyeler:** Bir sınıfın ya da nesnenin üyeleri, onun alanları ve metotlarıdır.
- **Sınıf:** Bir tiptir. `int` ya da `float` nasıl bir tipse, sınıf da öyle bir tiptir. Yani sınıf tanımlamak aslında yeni bir veri tipi tanımlamaktır.
- **Örnek (instance):** `0`, `C` sınıfının bir nesnesiyse `0`, `C`'nin bir örneğidir.

Hoca "örnek" sözcüğünün önemli olduğunu söyledi. Soyut bir şeyden nesne üreterek onu somutlaştırırsınız, ondan bir örnek çıkarırsınız ve bu örnek bellekte yer tutar.

5. Nesne Oluşturma ve Bellek

5.1 Nesne Bellekte Nerede Durur?

Hoca bir `Book` çizdi. Alanları `id`, `author`, `price` ve `numberOfPages` idi. Sonra bu sınıftan bir nesne üretti: adı Java olan, fiyatı 100 TL, sayfa sayısı 500 olan bir kitap.

Üretilen nesne bellekte yer tutar ve içinde erişebileceğimiz, bu örnekte 4 alan vardır. Yapılarda olduğu gibi bu alanlara nokta operatörüyle erişilir. Nesneler fiziksel olarak belleğin heap bölgesinde tutulur. Bize lazım olan ise heap'teki adresin, stack bölgesindeki değişkende tutulan referans bilgisidir. Bu yüzden nesnelere referans tipli veri denir. Hocanın kısa kuralı şudur: Java'da `new` ile oluşturulan her şey referans tiplidir. Fiziksel olarak heap'te saklanır, adresin değeri ise stack'te kullandığımız değişkene yazılır.

```
Book b;          // yalnızca bir referans değişkeni: henüz nesne yok
b = new Book();  // nesne heap'te oluşur, referansı b'ye yazılır
```

Hoca `b` değişkeninin stack bölümünde, gerçek nesnenin heap bölümünde durduğunu, buna referans mekanizması dendiğini açıkladı. Yalnızca `Book b;` yazmak nesne oluşturmak değildir.

> **Ek bilgi (derste anlatılmadı):** Hocanın anlattığı stack yerleşimi büyük olasılıkla metot içinde tanımlanan yerel değişkenler içindir. Bir referans sınıfın alanı olarak tanımlanırsa, büyük olasılıkla stack'te değil, ait olduğu nesneyle birlikte heap'te durur.

5.2 Bir Satırda İki Operatör

Nesne oluşturmak için `new` kullanılır ve `new` bir operatördür. Hoca sordu: Bu nesne oluşturma satırında kaç operatör kullanıldı, hangi sırayla çalıştı?

Soru: Kaç operatör kullanıldı? **Cevap:** İki operatör: `new` ve atama. Önce `new` nesneyi fiziksel olarak üretir. Üretilen nesne "havada kalmasın" diye onu bir yerde saklamak gerekir, çünkü ona `b` üzerinden erişeceğiz, içine değer yazıp okuyacağız. `new` bir şey döndürür: oluşan nesnenin referansını. Atama operatörü de bu referansı `b`'ye yazar.

> **Hocanın vurgusu:** Sıra çok önemlidir. Önce nesne oluşturulur, sonra yeni nesnenin referansı `b`'ye atanır. Hoca bunu ayrıca sorabileceğini söyledi.

Aynı sıra metot çağrısında da geçerlidir. Bir `f` metodu `Book` tipinde parametre alıyorsa, `new Book()` ifadesi önce nesneyi oluşturur, sonra referans metoda aktarılır.

```
class Book {
    Book() {
        System.out.println("1) yapici calisti");
    }
}

public class Test {
    static void f(Book b) {
        System.out.println("2) f metodunun icinde");
    }

    public static void main(String[] args) {
        f(new Book());
    }
}
```

Çıktı önce `1) yapici calisti`, sonra `2) f metodunun icinde` olur (javac ile denendi).

6. UML Ailesi ve Neden Kod Üreten Bir Araç

UML genel bir kavramdır ve altında hocanın hatırladığı kadarıyla 12-13 çeşit diyagram vardır. Hoca bu çeşitleri kaynağından okudu:

- **Yapısal diyagramlar:** Sınıf (class), nesne (object), bileşik yapı (composite structure), bileşen (component), paket (package), dağıtım (deployment). Dağıtım diyagramı, bir projenin kurulumuyla ilgili işlemleri gösterir.
- **Davranışsal diyagramlar:** Kullanım durumu (use case), etkinlik (activity), durum makinesi (state machine).
- **Etkileşim diyagramları:** İletişim (communication), sıralama (sequence), zamanlama (timing), etkileşime genel bakış (interaction overview).

Bu listede toplam 13 tür vardır (python ile hesaplandı: 6 + 3 + 4 = 13). Kullanım durumu diyagramı projelerde çok kullanılır: Programda bir doktor varsa hangi yetkilere sahip olacağı, bir metodu çalıştırdığında ne olacağı diyagram hâline getirilir.

Bu derste ağırlıklı olarak sınıf diyagramları üzerinde durulacak. Gerekirse diğerlerine de bakılacak. Hoca, diğer diyagramların sistem analizi ve yazılım mühendisliği derslerinde de anlatıldığını belirtti.

Kaynak olarak Deitel kitaplarındaki ATM örnek çalışmasını (ATM Case Study) önerdi. Bu örnekte önce projenin bütün hikâyesi verilir, buradan gereksinim analizi yapılır, sonra sınıfların ne olacağı belirlenir. Bir veri tabanı tasarlar gibi nesne tabanı tasarımı yapılır. Hocanın beğendiği bir başka kaynak da "Distilled UML" kitabıdır. UML'in sürümleri de vardır ve hoca bildiği kadarıyla şu anki sürümün 2.0 ya da 2.1 olduğunu söyledi.

> **Ders sonrası not:** Hoca sürümü kesin söylemedi. Güncel UML sürümü büyük olasılıkla 2.5 ailesindendir.

Soru: Slaytın başında önerilen Violet UML gibi bir uygulamayla aynı işi göremez miyiz, yoksa farklı bir amaç mı var? **Cevap:** Hocanın kazandırmak istediği şey, bu işin yazılım mühendisliği tarafına kaymaktır. Violet UML'de yalnızca çizim yapılır. Pek tavsiye etmediğini, isteyenin kullanabileceğini söyledi. Hocanın amacı, bir problem verildiğinde UML şemasını çizip kod üret (code generate) dediğinizde kod iskeletinin çıkması, sonra da içerideki metotları yazabilmenizdir.

Önemli olan, nesnelerin ne olduğunu ve aralarındaki ilişkinin nasıl olacağını kafanızda çözmektir. Kodlamaya geçmeden UML'i kafanızda kurup çizerseniz, kod üretmek basit bir iş olur.

> **Hocanın vurgusu:** Bu dersin amacı Java dilini öğrenmek değildir. Amaç, verilen bir problemi alıp nesnesel düşünmek ve o nesnelerin modellerini oluşturmaktır. Hoca, Teknopark'taki firmaların öğrencilerin tasarım kalıplarını bilmediğinden ve nesne yönelimli tasarımdan yoksun olduğundan şikâyet ettiğini aktardı. Bu ders tek başına yetmez, üzerine tasarım kalıpları ya da nesne yönelimli analiz ve tasarım dersi gerekir.

Etkileşim diyagramlarına dönersek, slayta göre iki türü vardır: sıralama (sequence) ve işbirliği (collaboration). Bu derste sıralama diyagramı çizilecek. "Etkileşim" adı diyagramın doğasını yansıttığı için etkileşim ve sıralama sözcükleri birbirinin yerine kullanılabilir.

> **Ders sonrası not:** Hoca slayttaki bu ifadeyi "birbiriyle farklı bir şekilde kullanabiliriz" diye çevirdi; doğrusu büyük olasılıkla "birbirinin yerine kullanabiliriz"dir. Ayrıca slaydın "iki tür" demesiyle hocanın az önce okuduğu dört türlü liste aynı değildir. İşbirliği diyagramı büyük olasılıkla listedeki iletişim diyagramının eski adıdır.

7. Her Nesne Ayrı Bir Bireydir

Her nesne, aynı tipten olsa bile ayrı bir bireydir. Aynı alanlara sahip iki nesne düşünelim. Bu alanların değerleri, yani durumları farklı olabilir, aynı da olabilir. Bu önemli değildir, çünkü iki nesne yine de farklı bireylerdir.

Hoca sebebini Türkçe açıkladı: Bellekte oturdukları yerler başkadır. İçlerindeki değerleri birebir aynı yapsanız bile farklı nesnelerdir. "Biraz daha işaretçi mantığıyla düşünün" dedi. Aynı duruma sahip iki nesne oluştursanız bile bunlar belleğin farklı bölgelerinde temsil edilir.

JVM bu amaçla tekil bir tanımlayıcı oluşturur, bu da referanstır. Bu işlem ve diğer bellek işlemleri programcıya görünmezdir. Öyle ki yordamsal programlamadaki işaretçi gibi araya giremezsiniz. Örneğin C'de `malloc` bellekte yer ayırır, işiniz bitince `free` ile bırakmanız gerekir. Java'da `free` ya da `delete` gerekmez. Java Sanal Makinesi bu bellek konumunu başkası kullanabilsin diye kendisi temizler.

Hocanın sokak örneği şudur: Sokakta giden herhangi iki arabanın model, renk ve plaka gibi özellikleri olur. Model ve renk farklı olabilir, aynı da olabilir. Aynı sarı Anadol'dan iki tane görseniz bile plakaları farklıdır. Sahte plaka olsa ve plakalar aynı çıksa bile sürücüleri farklıdır. Hocanın özeti şöyledir: Bir özellik aynı olsa öbürü farklı olabilir. Bütün özellikler aynı olsa bile bellekte oturdukları yerler farklıdır.

Bu yüzden aynı tipten iki nesne, aynı mesaja farklı cevaplar verebilir. Çünkü durumları farklı olabilir ve onlara farklı parametreler verilebilir.

8. Nesneye Mesaj Gönderme

8.1 Amaç: Bir İş Yaptırmak ya da Üyeye Erişmek

Bir nesneye mesaj göndermenin iki amacı vardır. Birincisi nesneye bir iş yaptırmaktır: kitapları yazdır, fiyat hesapla (`printBook`, `calculatePrice` gibi). Bu birer emirdir, birer mesajdır.

İkincisi nesnenin bir üyesine erişmektir: kimliği ya da kitabın adını almak, kitabın adını değiştirmek gibi. Bunun için nesnenin alanlarına erişmek gerekir. Burada iki genel metot türü öne çıkar: değer alan getter ve değer atayan setter. `getAuthor`, `setAuthor` birer örnektir. Bu ikisi temel işlemlerdir. NetBeans gibi tümleşik geliştirme ortamları bu metotları doğrudan üretir, yazmaya gerek kalmaz. Hoca bunları belki birkaç kez elle yazacaklarını, sonra doğrudan oluşturacaklarını söyledi. Önemli olan sınıfın hangi özelliklere sahip olacağına karar vermektir.

Üye metoda erişimin de temel amacı `calculatePrice` gibi bir metodu çalıştırmaktır. Bu çalıştırma isteğe bağlı parametrelerle yapılabilir: `printBook` metoduna bir argüman gönderilebilir.

8.2 Parametre Uyumu

Metot çağırmak C'deki fonksiyon çağrısına benzer. Ama bir farkı vardır: Nesnenin metodu, aksi söylenmedikçe o nesnenin üyeleriyle çalışır. "Aksi" nasıl olur sorusunun cevabı, nesneler arasındaki farklı ilişki türleri öğrenilince görülecektir.

C'de `int` parametreye `int` ya da dönüşebilen bir tip gönderiliyordu. Burada da gönderilen parametre uyumlu olmalıdır. Ama bu uyum, illa aynı nesne türü olmak anlamına gelmez. Alt-üst ilişkisi gibi farklı ilişkiler tanımlanacak ve burada kullanılacaktır. Erişim ise nokta operatörü ya da başka yollarla sınıfın üye değişken ve metotlarına ulaşmaktır.

8.3 Etkileşim Diyagramı: client ve kit

Nesne yönelimli terminolojide bir nesne, başka bir nesnenin üyesine ona mesaj göndererek erişir. Nesne yönelimli bir program, nesneler arasında mesaj alışverişiyle çalışır. Slayttaki örnek şu adımlardan oluşur:

1. `client` adında bir nesne vardır. Bu nesnenin sınıfı önemsizdir.
2. `kit` adında bir nesne vardır ve sınıfı `Car`'dır. Gösterimi `kit : Car` biçimindedir, iki nokta işareti bu UML gösteriminde zorunludur.
3. `Car` sınıfında `getPlate` adında bir metot vardır. Bunu mesajın bu sınıfta tanımlı olmasından anlarız.
4. `client`, `kit` nesnesine `getPlate()` mesajını gönderir. Bu normal bir fonksiyon çağrısı gibidir: "Plaka numarasını öğrenmek istiyorum."
5. `kit`, plakasını cevap olarak döndürür.

Cevabın tipi tasarıma bağlıdır. `void` olabilir, ya da `int`, `int` dizisi, `String`, `String` dizisi veya araba ya da plaka gibi özel bir tip olabilir. Hoca, C'de dizi döndürülemezken burada döndürülebildiğini söyledi.

Bu UML gösteriminin Java karşılığı şudur. `kit` bir nesne, nokta bir operatör, `getPlate` ise `Car` sınıfındaki bir metottur.

```
kit.getPlate();
```

9. Paketler

9.1 Neden Paket Kullanılır?

Bir projeyi mantıksal parçalara, yani paketlere bölebiliriz. Örneğin `ClassA` ve `ClassB` `package1` içinde, `ClassC` ise `package2` içinde olabilir. Amaç, tasarımı mantıksal olarak ayırmak ve veri tipleri arasındaki çakışmaları önlemektir. Farklı sınıfların içinde aynı adlı bir tip bulunabilir, paketler bunları birbirinden ayırır.

Hoca sebebini Türkçe açıkladı: Projede aynı adlı sınıflar ya da üyeler olabilir, ama işlevleri farklıdır. Doktor ve muhasebeci için kendilerine ait bilgi görüntüleme ve listeleme sınıfları düşünelim. Doktorun bilgi listelemesi ile muhasebecinin bilgi listelemesini paketlerle ayırabiliriz.

Hocanın daha çok beğendiği örnek bir yazılım tasarımıdır. Muhasebe departmanını ilgilendiren paketler bir yerde, doktorların hastalarıyla ilgili işleri başka bir pakette durur. Hoca, bir paketten diğerine doğrudan erişilemediğini ve bunun bir anlamda güvenliği artırdığını söyledi. Paketler soyut kaplardır (abstract containers). Hem gruplama yapar hem de yetkilendirmeyi sağlar.

> **Ders sonrası not:** "Bir paketten diğerine doğrudan erişilemez" sözü yalnız `public` olmayan üyeler için geçerlidir. Başka paketteki `public` sınıf `import` ile kullanılabilir. Görünürlük belirtilmeyen üyeye başka paketten erişmek ise derleme hatası verir: `pk is not public in A; cannot be accessed from outside package` (javac ile denendi).

9.2 import

UML'deki paket gösteriminin Java karşılığı `import`'tur. C'deki dosya eklemeye benzer şekilde kullanılır.

```
import package1.ClassA;      // package1 içinden tek bir sınıf
import package1.*;           // package1 içindeki bütün sınıflar
import package1.package2.*;  // package1 içindeki package2'nin bütün sınıfları
```

Farklı paketlerdeki sınıflar aynı adı taşıyabilir. Slayttaki örnek `File` sınıfıdır. Biri `java.io` içindedir, öbürü başka bir paketin içinde. Hoca sonuçta her şeyin sıfır ve birlerle yazıldığını, ama program yazarken bu iki `File` sınıfını karıştırmamak gerektiğini söyledi.

`package1` import edildiğinde yalnızca o paketin doğrudan içindeki sınıflar gelir, `package2`'nin sınıfları eklenmez. Bunları ayrıca eklemek gerekir (javac ile denendi: yalnız `import package1.*;` yazılınca `package2`'deki sınıf için `cannot find symbol` hatası alınıyor, ikinci `import` eklenince derleniyor). Hoca bunun özellikle GUI uygulamalarında kullanılacağını söyledi.

10. Nesne Diyagramı ve Easy UML

Aradan sonra hoca UML gösterimine döndü ve daha önce söylemeyi unuttuğu bir kuralı ekledi. Nesne diyagramındaki yazı altı çizili olur: önce nesnenin adı, sonra sınıfın adı yazılır, araya iki nokta konur. Sonraki aşamada sınıflar arasına ilişkileri gösteren çizgiler çizilecek. Bu ilişkilerin de farklı türleri vardır.

> **Hocanın vurgusu:** Nesnenin ve sınıfın nasıl yazıldığına dikkat edin. Hoca, UML çizmenin ya da çizilmiş bir UML'in boş yerlerini doldurmanın sorulabileceğini söyledi.

Nesneler arasındaki hareketi gösteren diyagram ise etkileşim diyagramıdır: Bir nesne diğerinden servis alır, işlem yaptırır. Paketlerin de kendi diyagramları vardır.

Hoca NetBeans'te Easy UML'i açarak gösterdi. Araçta sınıf, arayüz (interface) ve numaralandırma (enumeration) oluşturma ile bunların arasındaki ilişkileri kurma seçenekleri vardır. Sınıf ekleme, ilişki ekleme, diyagramı düzenleme ve renk gibi komutlar görülür. Bu araçta paket ekleme seçeneğinin olmadığını gördü. Paket destekleyen araçlarda önce paket eklenir, sonra sınıflar onun içine konur ve ilişkiler tanımlanır.

Terminolojide çok fazla şekil yoktur. Asıl zorluk başka yerdedir: bir senaryoyu anlayıp ondan tasarım çıkarmak.

> **Hocanın vurgusu:** Tasarımda ileriyi düşünmek gerekir. Yazarın adını soyadını şimdilik kitabın içine gömerseniz, iki gün sonra yazarla ilgili başka bir bilgi geldiğinde tasarım "patlar". Bunu önceden düşünüp yazarı ayrı bir nesne olarak tasarlamak gerekir.

11. Bilgi Gizleme

11.1 Günlük Hayattan Sezgi

Görünürlük kuralları ya da bilgi gizleme, nesne yönelimli programlamanın önemli kavramlarıdır. Günlük hayatta bilgisayar, monitör, cep telefonu gibi birçok nesne kullanırız. Hepsinin özellikleri ve işlevleri vardır, ama biz bunların yalnızca bir kısmını kullanırız.

Monitör örneğinde LCD tüp, yansıtma ya da renk karışımı birer işlevdir, ama bunları doğrudan kullanmayız. Monitörü açmak için üzerindeki düğmeye basarız. İç işlevlere erişmemize gerek yoktur. Dahası bu erişim kullanıcıya yasaklanmalıdır, çünkü kullanıcı bunlara erişirse işlevleri kolayca bozabilir.

Hocanın lamba örneği şudur: Lambayı açarken arka planda olan şey elektrik kablolarının birbirine değmesidir. Ama gidip elimizle değdirmeye çalışırsak her şeyi berbat eder, kendimize de zarar veririz. Amaç lambayı yakmak ve devreyi tamamlamaktır. Elektrik bir nesne, lamba bir nesnedir. Aralarında görev yapan bir anahtar nesnesi vardır. Buradan çıkan sonuç şudur: Nesnelerin her türlü özelliğine ve arka plandaki her nesneye erişmemiz gerekmez, hatta bu yasaklanmalıdır.

Aynı şey yazılımda da geçerlidir. Windows'un birçok uygulaması vardır ve içinde birbiriyle iletişim hâlinde nesneler çalışır. Herhangi bir nesneye erişip işlevini ya da değerini değiştirsek işletim sistemine zarar verebiliriz.

11.2 İlke

Bir nesne, aynı sınıf tipinden olan bütün üye ve metotlara erişebilir. `Book` bir sınıf, Java adlı kitap bir nesneyse, bu nesne `printAll`, `getID`, `getName` gibi metotlarına doğrudan erişir.

Ama nesnenin bazı üye ve metotlarını, farklı tipten nesneler erişemesin diye gizleyebiliriz. Görünürlük kuralları ya da bilgi gizleme budur. Hoca bunun dil açısından isteğe bağlı olduğunu, nesne yönelimli tasarım kuralları açısından ise kullanmak zorunda olduğumuzu söyledi. İlke şudur: Nesnenin iç işleyişiyle ilgili üyeleri, farklı tipten nesnelerden gizleriz. Böylece bir nesnenin, başka bir nesneyi kullanmak için onun iç ayrıntılarını bilmesi gerekmez. Örneğin bir kitap yazarın referansına sahiptir, ama yazarın adına ve soyadına doğrudan erişmez. Aynı şekilde `Author` sınıfının da `Book` sınıfının üyelerine doğrudan erişmesi gerekmez.

Slayttaki iki örnek de bunu anlatır:

- **Televizyon:** Televizyon izlemek için kumandadaki güç, ses ve kanal tuşlarının evrensel işaretlerini bilmek yeter. Televizyonun içinde katot tüpü olduğunu bilmeniz gerekmez. Dahası kullanıcıların LCD ya da plazma gibi yeni teknolojilerle üretilen televizyonlar için yeniden eğitilmesi gerekmez.
- **Borç:** Arkadaşınız sizden borç istiyor. Cüzdanınızı açıp verirsiniz ya da vermezsiniz. Maaşınızı ya da PIN numaranızı söylemek zorunda değilsiniz.

Hocanın özeti şudur: Modellemede gerekli alanları `private`, `public` gibi anahtar kelimelerle dış dünyaya kapatacağız.

12. UML Sınıf Diyagramının Yazımı

Daha önce boş bir sınıf görmüştük. Artık kutuya iki bölüm daha eklenir. En üstte sınıfın adı, ikinci bölümde alanlar, üçüncü bölümde metotlar yazılır.

```
+----------------------------------------+
| ClassName                              |
+----------------------------------------+
| - field : Type                         |
+----------------------------------------+
| + method() : ReturnType                |
| + method(x : int, y : float, z : Book) |
+----------------------------------------+
```

Bu kutudaki her işaretin bir anlamı vardır:

- **`-`** Java'daki `private` anahtar kelimesidir.
- **`+`** `public` anahtar kelimesidir.
- **`#`** `protected` anahtar kelimesidir. Java'da kalıtımla ilgilidir ve ileride görülecektir.
- **`~`** Paket görünürlüğüdür. Java'da varsayılan olarak gelen kuraldır, sonra görülecektir.

Metotlar parantezlerinden anlaşılır. Yazım kuralları şöyledir:

- Metot `void` döndürüyorsa dönüş tipi eklenmez.
- Metot bir değer döndürüyorsa, iki nokta konur ve dönüş tipi yazılır: `int`, `Book` ya da herhangi bir sınıf adı.
- Metot parametre alıyorsa, her parametre için önce ad, sonra iki nokta, sonra tip yazılır: `x : int`, `y : float`, `z : Book`.

  Ders sonrası not: Hoca `void` metoda dönüş tipi eklemenin yasak olduğunu söyledi. UML'de `void` yazılmaması büyük olasılıkla yaygındır, ama `: void` yazımı da büyük olasılıkla yanlış sayılmaz.

Dikkat edilecek nokta, bu yazımın Java ve C'ye uymamasıdır. C'de `int x` yazarken UML'de bunun tersi yazılır: `x : int`.

Bazen ek işaretler de kullanılır. `abstract` anahtar kelimesi eklenirse sınıf soyut sınıf olur. Ayrıntıları sonraki haftalarda görülecektir. Metotlara da `abstract` eklenebilir. Altı çizgi ise `static` demektir: Bir metot `static` ise adının altı çizilir.

Hoca, bu kutu verildiğinde onu koda çevirmeyi şöyle anlattı. Sınıf adı `public class ClassName` satırına ve onu izleyen süslü parantezlere karşılık gelir. `-` ile yazılan alan `private` tanımlanır. Metotların içi istenirse, o metoda uygun algoritma yazılır. Nesne yönelimli programlama yapısal programlamayı kapsar: Yapısal kısım aslında bu metotların içeriğinin tanımlanmasıdır, geri kalanı nesne yönelimli kısma girer.

13. Bilgi Gizlemenin Uygulanması

13.1 İlke Kusursuz Uygulanamaz

Uygulamada bilgi gizleme kusursuz biçimde uygulanamaz. Bir sınıfın kodundaki değişiklik yalnız o sınıfı değil, başka sınıfları da etkiler. Örneğin bir üyeyi `public`'ten `private`'a çevirirseniz derleme hatasıyla karşılaşırsınız.

> **Ders sonrası not:** Hoca bunun tersinin de ("or vice versa") derleme hatası vereceğini söyledi. Javac ile denendi: Üye `public`'ten `private`'a çevrilince başka sınıftaki erişim `x has private access in A` hatası veriyor. `private`'tan `public`'e geçiş ise hata vermiyor.

Bu ilkeye ne kadar uyarsanız, değişikliği tamamlamak için gereken kodlama yükü o kadar azalır, çünkü etkilenen sınıf sayısı düşer. Amacımız bu sayıyı azaltmaktır. Projenin sonraki aşamalarında ortaya çıkacak sonuçları önceden düşünmek gerekir.

13.2 Kural: Alan private, Erişim public

İlkeye uymak için üye alanlar `private` tanımlanır. Gerekli erişim metotları ise `public` tanımlanır. Erişim metotları (access methods), `private` üyelere erişmeye yarayan yardımcı metotlardır.

> **Hocanın vurgusu:** Bir kod yazmanız istendiğinde UML verilmediyse, o sınıfın üyelerini `private` olarak düzenleyin. Bunu yapmazsanız ileride birçok şey değişir. Baştan `private` yapmak en güzelidir.

`private` yapmak değişkene erişilemeyeceği anlamına gelmez. Yalnızca doğrudan erişim kapanır. `set` ve `get` metotlarıyla yine erişilebilir. Setter bir üyenin değerini değiştirir, getter ise değerini verir: `setName`, `getName` gibi.

13.3 Car Sınıfının UML Gösterimi

Slayttaki genel örnek `Car` sınıfıdır:

```
+----------------------------+
| Car                        |
+----------------------------+
| - plate : String           |
+----------------------------+
| + getPlate() : String      |
| + setPlate(plate : String) |
+----------------------------+
```

Sınıfın adı `Car`'dır. Alanın adı `plate`, tipi `String`, erişim belirleyicisi `private`'tır. `getPlate` ve `setPlate` `public` metotlardır. Adlarından anlaşıldığı gibi `getPlate`, `plate` değişkeninin değerini almaya, `setPlate` ise bu değişkene `String` bir değer atamaya yarar.

Burada değişiklik yapmak kolaydır. Erişimi kısıtlamak istiyorsak `setPlate` metodunu koddan kaldırabiliriz ya da görünürlüğünü `private` yapabiliriz. Hoca, silmenin her zaman makul bir çözüm olmadığını, erişilebilirlik işaretlerinin kullanılacağını söyledi.

14. Static Alanlar

14.1 Ortak Bir Alan Paylaşmak

Aynı sınıftan olsalar bile her nesnenin durumu farklıdır. `Book` bir sınıfsa, `b1 = new Book()` ve `b2 = new Book()` nesneleri kitapla ilgili birçok değişken içerir ve durumları farklıdır.

Bazı durumlarda ise belli bir tipteki bütün nesnelerin ortak bir alanı paylaşması gerekebilir. Kantindeki ya da bankadaki sırada müşteri sayısı buna örnektir. Bu sayı bütün kullanıcılar için ortak bilgidir, ama kullanıcılar birbirinden farklıdır. Sınıfta ortak bir alan paylaşmak için `static` anahtar kelimesi kullanılır.

```
static int numberOfCustomers;
```

Normalde üyelere nesne değişkeni üzerinden erişilir: `b1.name`, `b1.id` gibi. `static` üyelere ise sınıf adı üzerinden erişilir: `Book.memberName` gibi. Nesne değişkeni üzerinden de (`b1.memberName`) erişmek mümkündür, ama sınıf adı tercih edilir. Javac ile denendi: Değer `Book.memberName` üzerinden değiştirildikten sonra `b1.memberName`, `b2.memberName` ve `Book.memberName` üçü de aynı değeri veriyor.

14.2 Kantin Örneği

Hoca ortak bilgiyi şöyle örnekledi. Bir kantine girdiniz. Kantinde tabakları toplayan görevliler var. O gün kantin 1000 kişiye hizmet verecek, bininci müşteriden sonrasını kabul etmeyecek. Son müşteri geldi diye bir bayrak (flag) vardır. Bininci müşteriyi alan görevli bu bayrağı değiştirince diğer görevliler de bunu görür ve işi bırakabileceklerini anlar. Görevliler birbirinden farklı nesnelerdir, ama hepsi bu ortak alana bakarak karar verir.

Bir başka örnek numaratördür: `static` bir numaratör herkes için aynıdır. Hoca bunu C'deki `static` yerel değişkene benzetti. C'de fonksiyon içinde `static` tanımlanan değişken, fonksiyon tekrar tekrar çağrılınca kaldığı yerden devam ediyordu. Burada ise birbirinden bağımsız nesneler aynı alanı görür. `b1` ile `b2`'nin adı, kimliği ve bellekteki yeri farklıdır, ama `static` alan hepsinde aynıdır. Çünkü `static` alan sınıfın bir üyesidir. `static` olmazsa nesnenin üyesi olur.

Soru: Sınır dolunca sistemi nasıl kapatacağız? **Cevap:** Bu basittir. Yeni öğrenci ekleme diye bir fonksiyon vardır. Bu fonksiyon, eklemeyi yapmadan önce bir `if` ile bayrağa bakar. Bininci öğrenci geldiyse sistem kapanmıştır ve yeni öğrenci kabul edilmez.

Soru: Bu sadece görevliler sınıfına ait olmayabilir mi? **Cevap:** Tabii. Üç sınıf düşünelim: öğrenci, kantin (ya da banka) ve çalışanlar. Üçü birbiriyle haberleşecek. Öğrenci ekleme metodunu nereye yazmalıyız? Kantine, çünkü öğrenci kendi kendini sisteme ekleyemez. Bu yüzden değişken kantinde `static` olarak tutulur ve görevliler onu değiştirir. Öğrencilerden ise bu değişken gizlenir. Hoca iş parçacıkları (thread) anlatılınca bununla ilgili bir örnek kodlayacağını söyledi.

`static` için son örnek bilinen ortak özelliklerdir. Her otomobilin 4 tekerleği vardır. Hoca özellikle "otomobil" dediğini, daha büyük araçları kastetmediğini belirtti. Kısacası herkese ortak bilgi paylaşımı gereken bir durum varsa `static` kullanılır.

15. Static Metotlar

Aynı tipten iki farklı nesne aynı mesaja farklı cevap verebilir. Ama bazı durumlarda belli bir tipteki bütün nesnelerin ortak bir davranışı paylaşması gerekebilir. Bu durumda metot `static` anahtar kelimesiyle tanımlanır. Bu metotlara sınıf adı üzerinden erişilir (`ClassName.method`) ve nesne oluşturmaya gerek yoktur.

Hoca buna örnek olarak `System.out.println` metodunu verdi: Çok kullanılır ve farklı amaçlarla kullanılabilir.

> **Ders sonrası not:** `println` `static` bir metot değildir. Javac ile derlenip çalıştırılan yansıma (reflection) denetiminde `PrintStream.println` için `static` sonucu `false`, `System.out` alanı için `true` çıktı. Yani sınıf adıyla erişilen `static` üye `System.out` alanıdır. `println` ise bu alanın gösterdiği `PrintStream` nesnesinin metodudur.

`static` bir metodun içinde yalnızca o sınıfın `static` üyeleri kullanılabilir.

> **Ders sonrası not:** Hoca “static üyeleri static olmayan bir metotta kullanırsanız derleme hatası alabilirsiniz” dedi. Doğrusu tersidir (javac ile denendi). `static` olmayan metot `static` alana sorunsuz erişiyor. `static` metot `static` olmayan alana doğrudan erişince `non-static variable x cannot be referenced from a static context` hatası alınıyor. `static` metot içinde bir nesne oluşturulup alana o nesne üzerinden erişmek ise derleniyor; `main` içinde yapılan da budur.

16. Final Alanlar ve Metotlar

Bir üyenin değerinin sabit kalması gerekebilir. Bu, C’deki `const` tanımına benzer. Bu durumda üye Java’da `final` anahtar kelimesiyle tanımlanır, UML’de de `final` ile belirtilir.

`final` üyeye bir nesne için yalnızca bir kez değer atanabilir. Bu atama genellikle nesne oluşturulurken yapılır. Yani `Book b = new Book()` yazıldığında, atama nesne oluşturmanın arkasında doğrudan çalışır. Slayttaki örnek şasi numarasıdır: Arabanın şasi numarası fabrikada üretilirken üzerine kazınır ve sonradan değiştirilemez. Javac ile denendi: `final` alana ikinci kez atama `variable x might already have been assigned`, yapıcı dışındaki bir metottan atama `cannot assign a value to final variable c` hatası veriyor.

`final` metot ise override işlemini engellemek için kullanılır. Bu konu kalıtımla ilgilidir ve ayrıntıları sonra görülecektir.

> **Ders sonrası not:** Hoca Türkçe özetinde `final` metotların “kalıtımla türetilemez” hâle geldiğini de söyledi. Javac ile denendi: `final` metot alt sınıfa geçiyor ve alt sınıf nesnesiyle çağrılabiliyor. Engellenen yalnız override işlemidir (`m() in B cannot override m() in A`).

Bir metot ya da alan tanımında `static` ile `final` birlikte kullanılabilir. Metot hem override edilemez hem de nesne oluşturmadan sınıf adıyla çağrılır. Hoca örneği `public static final` ve `list book` diye söyledi. Dönüş tipini söylemediği için aşağıda `void` yazıldı (ara adım eklendi, javac ile denendi).

```java
public static final void listBook() {
    System.out.println("listBook");
}
```

Özetle `final` tek seferlik tanımla, `static` ise ortak kullanımla ilgilidir. UML sınıf şemasında üye `final` ve `static` sözcükleriyle belirtilir, `static` ayrıca altı çizgiyle gösterilir.

17. Yapıcı Metotlar ve Varsayılan Değerler

17.1 Özel Amaçlı Bir Metot

Nesne oluşturulurken nesne yönelimli kavramlarla ilgili bir olay gerçekleşir: yapım (construction). Bu sırada özel amaçlı bir metot, yani yapıcı (constructor) çalışır. Yapıcının özellikleri şunlardır:

- Adı sınıfın adıyla aynıdır. `public class Book` için yapıcının adı `Book`’tur.
- Dönüş tipi yoktur, `void` de yazılmaz.
- Parametre eklenebilir.
- Programcı yazarsa nesne oluşturulurken açıkça (explicitly) çalışır. Yazmazsa örtük (implicit) bir yapıcı devreye girer.

  Ek bilgi (derste anlatılmadı): Sınıf adıyla aynı adı taşıyan bir tanıma `void` yazılırsa o tanım yapıcı olmaz, sıradan bir metot olur. Javac ile denendi: `void Car()` tanımlı sınıfta `new Car()` bu metodu çalıştırmıyor, metot ancak `c.Car()` diye açıkça çağrılınca çalışıyor.

Yapıcılar özellikle üyelere ilk değer atamak için kullanılır. Bir kitap nesnesi oluşturulurken kimlik, başlık, fiyat ve sayfa sayısı nesneye aktarılabilir. `new Book(1, "Java", ...)` çağrısındaki argümanlar, yapıcı içinde `Book` sınıfının üyelerine aktarılır.

Hocanın Türkçe anlatımıyla: Bir nesne üretir üretmez değişkenlerin oraya aktarılmasını istiyorsam yapıcı kullanırım. Bu işi yapıcıyla yapacaksak, bunun için ayrıca setter ve getter

gerekmez. Argümanların sırası önemlidir, sıra değişince başka bir şey olur. Yapıcı, nesne oluşturulduğunda arka planda olacak olayları tanımlamamızı sağlar. Tanımlamazsak arka planda varsayılan (default) yapıcı devreye girer.

17.2 Varsayılan Değerler

Hoca `Book` sınıfını şu üyelerle yazdı ve varsayılan yapıcının ne yaptığını sordu:

```
class Author {
}

class Book {
    int id;
    int numberOfPage;
    float price;
    String name;
    Author author;
}

public class Test {
    public static void main(String[] args) {
        Book k = new Book();
        System.out.println(k.id + " " + k.numberOfPage + " " + k.price
                + " " + k.name + " " + k.author);
    }
}
```

Bu kodun çıktısı `0 0 0.0 null null` olur (javac ile denendi). Değer tipli üyede varsayılan değer tipe göre değişir: `int` için 0, `float` için 0.0, `boolean` için `false`. Referans tipli üyelerde varsayılan değer `null`'dır. Yani `name` ve `author` `null` olarak başlar.

> **Ders sonrası not:** Hoca `char` için önce "null", ardından "sıfır gibi" dedi. `char` değer tiplidir. Javac ile denendi: `(int) k.c` sonucu `0` çıkıyor. `null` yalnız referans tipli alanlar içindir.

> **Ders sonrası not:** Hoca varsayılan değerleri varsayılan yapıcının atadığını söyledi. Javac ile denendi: Alanlara hiçbir şey atamayan parametreli bir yapıcı yazıldığında da alanlar `0` ve `null` çıkıyor. Yani bu değerler yalnız varsayılan yapıcıya bağlı değildir.

18. Sonlandırıcı Metot ve Çöp Toplayıcı

18.1 finalize

Nesne oluşturulduğunda yapıcı çalışır: Tanımlıysa açıkça, tanımlı değilse örtük olarak. C'deki gibi nesnenin bellekten bırakılması konusu da vardır. Nesne yok edilirken bazı işlemler yapmak için sonlandırıcı (finalizer) metot kullanılır. Bu metot, nesne yok edileceği zaman Java Sanal Makinesi tarafından örtük olarak çalıştırılır. Hoca, C++ ya da C#'ta sonlandırıcıyı kendi kodunuza açıkça ekleyebileceğinizi söyledi.

Metodun adı `finalize`'dır, parametre almaz ve doğrudan bir şey döndürmez. İçine ne istersenız yazabilirsiniz. Hocanın örnekleri, "Object destroyed" ya da "program is closing" gibi bir mesaj yazdırmak ve yok edilirken bazı nesneleri kopyalamaktır.

```
protected void finalize() {
    System.out.println("Object destroyed");
}
```

18.2 Bellek Yönetimi

C ve C++'tan farklı olarak Java programcıları bellek yönetimiyle çoğunlukla uğraşmaz. Hoca bu cümleden ne anladığımızı sordu.

Soru: “Bellek yönetimiyle uğraşmak” ne demektir? **Cevap:** Öğrenciler `free` işlemini, dinamik bellek ayırmayı, `calloc`, `malloc` ve `free` çağrılarını söyledi. Hoca hepsinin doğru olduğunu belirtti.

Bellekten yer ya stack bölgesinden ya da heap bölgesinden ayrılır. Heap’te yapılan bellek tahsisini, işletim sistemiyle yapılan özel izinler gibi düşünebilirsiniz, çünkü bu bölge dinamik bir alandır. C’de bu alandaki nesneyle işimiz bitince belleği fazla yormasın diye hemen `free` ediyorduk. Hoca C++’ta bunun adının `delete` olduğunu söyledi.

Java’da ise bir nesne “başıboş” dolaşıyorsa, yani bir referansa atanmamışsa ya da bir yerde kullanılmıyorsa, Java Sanal Makinesi’nin çöp toplayıcısı (garbage collector) zaman zaman devreye girer ve kullanılmayan nesnelerin yerlerini yeniden kullanılabilir hâle getirir.

Hocanın adım adım örneği şudur:

1. `Book b = new Book();` satırıyla bir nesne oluşur ve ona `b` üzerinden erişilir.
2. `b` üzerinden bazı işler yapılır.
3. `b = new Book();` satırıyla ikinci bir nesne oluşur, referansı `b`’ye atanır.
4. Artık `b` ikinci nesneyi gösterir. İlk nesneyi gösteren bir referans kalmamıştır, başıboş bir nesne gibidir.
5. Java Sanal Makinesi bu nesneyi kendisi temizler.

18.3 Çöp Toplayıcıyı Tetiklemek

Slayta göre sonlandırıcı metotlar bu derste daha fazla işlenmeyecek. Hoca en fazla şunu anlatacağını söyledi: Sonlandırıcıyı elle çalıştırıp çöp toplayıcıyı tetikleyebiliriz. Kendi deneyiminden C# örneği verdi. Görüntü işleme uygulamalarında çöp toplayıcı çalışana kadar zaman geçiyor, bu sırada bilgisayar çok kasılıyordu. Bu yüzden bir algoritma tamamlanır tamamlanmaz çöp toplayıcıyı kendisi tetikliyordu. Hoca, C++’ta buna yıkıcı (destructor) dendiğini de söyledi. Sonlandırıcının içinde çöp toplayıcı çağrılırsa başıboş nesne bellekten hızlıca silinebilir.

> **Ders sonrası not:** Javac ile denendi: `b.finalize()` elle çağrıldığında sıradan bir metot gibi çalışıyor, mesajı yazdırıyor ve `b` nesnesi kullanılmaya devam edebiliyor. Yani bu çağrı nesneyi yok etmiyor. Java’da çöp toplama isteği büyük olasılıkla `System.gc()` ile yapılır ve JVM’nin bunu hemen yerine getireceği büyük olasılıkla garanti değildir. Ayrıca JDK 25’te `Object.finalize` için `since=9, forRemoval=true` işareti okundu; javac bu metodu `[removal]` uyarısıyla karşılıyor.

19. Yapıcı Kuralları ve Örnekleme

19.1 Kurallar

Slayttaki kurallar şunlardır:

- Yapıcılar `public`’tir, çünkü yeni nesne oluşturmak için bu metoda erişmek gerekir.
- Adları sınıfın adıyla aynıdır.
- Nesne oluşturmak için kullanılsalar da gövdelerinde `return` komutu verilmez ve dönüş tipleri yoktur. Doğrudan `public Car` yazılır. Ek parametre gerekiyorsa tanıma eklenir.
- `final` alanlara değer atamak için en uygun yerdir.
- `new` anahtar kelimesiyle birlikte kullanılırlar. `new` bu metodu doğrudan tetikler.

> **Ders sonrası not:** Yapıcılar `public` olmak zorunda değildir. Hoca da aynı derste `PrintStream` sınıfında `private` bir yapıcı gösterdi ve bunun engel olmadığını söyledi. Javac ile derlenen yansıma denetiminde JDK 25’te `PrintStream` sınıfının iki `private` yapıcısı listelendi.

19.2 Örnekleme

Örnekleme (instantiation), yapıcının çalıştırılmasıyla ilgili bir işlemdir. Bir nesneyi tanımlamak onu kullanmak için yeterli değildir, yapıcısını çalıştırarak örneklemek gerekir. `Car kit;` yalnızca tanımlamadır, burada somut bir nesne yoktur. `new` yazılınca yapıcı tetiklenir ve nesne gerçeklenir.

```
kit = new Car();
```

Slayttaki diyagramda başka bir sınıfın bir metodu bu işlemi yapar. Yapıcı çalışır ve yeni nesnenin referansı `kit` değişkenine atanır. Hoca burada belleğin hangi kısmının kullanıldığını sordu.

Soru: `kit` belleğin hangi bölümündedir? **Cevap:** `kit` stack bölümündedir, `new` ile gelen nesne ise heap bölümündedir. `kit` aslında nesnenin referansını tutar.

Slaytta aynı işlemin UML ve kod gösterimi iki biçimde verilmiştir: `kit` bir sınıfın üye alanı olarak ya da bir metodun geçici (yerel) değişkeni olarak. İkisi de yapıcı metodu çağırır.

19.3 Varsayılan Yapıcı ve Parametreli Yapıcı

Java'da yapıcı tanımlamazsanız parametresiz bir varsayılan yapıcı örtük ve otomatik olarak tanımlanır. Yapıcıya ek parametreler eklerseniz parametreli yapıcı kullanılır. `new` ile nesne oluştururken verdiğiniz argümanlar, yapıcı tanımındaki parametrelere atanır.

Programcı parametreli bir yapıcı yazarsa varsayılan yapıcı Java tarafından otomatik oluşturulmaz, yani iptal olur. Javac ile denendi: Yalnız `Car(String s)` yapıcısı olan sınıfta `new Car()` yazılınca `constructor Car in class Car cannot be applied to given types` hatası alınıyor.

> **Ders sonrası not:** Hoca bir yerde "constructor is explicitly defined" dedi. Bağlama göre kastedilen, yapıcının örtük (implicit) tanımlanmasıdır. Aynı şekilde "explicitly called" sözü de büyük olasılıkla "parametreli yapıcı açıkça yazıldıysa" anlamındadır.

Hangisinin kullanılacağı tasarımınıza bağlıdır. Burada iş mantığı (business logic) önemlidir: Probleminiz parametreli yapıcı gerektiriyorsa onu tanımlarsınız, gerekmiyorsa yazmanız gerekmez.

20. Aşırı Yükleme

20.1 Yapıcıları Aşırı Yüklemek

Yeni bir kavram da aşırı yüklemedir (overloading). Bir sınıfın yapıcılarını ve metotlarını aşırı yükleyebiliriz. İmzaları (prototype) farklıysa bir sınıfta birden çok yapıcı tanımlanabilir. Slayttaki örnek şöyledir (javac ile denendi):

```java
public class Car {
    public Car(int i) {
    }

    public Car(String s) {
    }

    public Car(int i, String s) {
    }
}
```

20.2 Metotları Aşırı Yüklemek

Aşırı yükleme metotlar için de kullanılır. Dairenin, dikdörtgenin ya da karenin alanını hesaplamayı düşünelim. Hepsine "şeklin alanını hesaplamak" denebilir, ama gerçekleştirmeleri farklıdır. Hoca, tek parametreli `calculateArea` ile karenin, iki parametreli `calculateArea` ile dikdörtgenin alanının hesaplanabileceğini söyledi. Metot adı aynıdır, imzalar farklıdır. Metot gövdeleri derste söylenmediği için aşağıda alan formülleriyle yazıldı (ara adım eklendi, javac ile denendi).

```java
public class Shape {
    public int calculateArea(int r) {
        return r * r;
    }

    public int calculateArea(int x, int y) {
        return x * y;
```

```
    }
}
```

Hocanın vurgusu: Aşırı yükleme çok kullanılan bir kavramdır ve genellikle override ile karıştırılır.

Ek bilgi (derste anlatılmadı): Aşırı yüklemede aynı sınıfta aynı adla farklı imzalar yazılır. Override işleminde ise alt sınıf, üst sınıftaki metodu aynı imzayla yeniden tanımlar. Javac ile denendi: `final` olmayan metot alt sınıfta override edilebiliyor ve alt sınıf nesnesinde yeni sürüm çalışıyor.

20.3 İmza Nasıl Farklılaşır?

Aşırı yükleme, bir metodun ya da yapıcının sınıf içinde farklı imzalara sahip olmasıdır. Önemli olan, metot adından sonraki kısmın farklı olmasıdır. Metot adından önceki kısmı değiştirmek yetmez: Aynı metodu `int` yerine `float` döndürecek şekilde yazmak olmaz. Javac ile denendi: Yalnız dönüş tipi farklı iki metot `method m(int) is already defined` hatası veriyor.

Hoca sınıfa, parametrelerin farklılığının nasıl sağlanabileceğini sordu.

Soru: Parametreler nasıl farklılaştırılır? **Cevap:** Tipleri değiştirilebilir, sıraları değiştirilebilir, sayıları değiştirilebilir. Örneğin `int`, `float` ve `String` parametreli bir metotta bu üçünün sırası kendi içinde değiştirilirse aşırı yükleme yapılmış olur (javac ile denendi). Biri tek, öbürü iki parametre alırsa yine imza değişir.

20.4 println Örneği

Hoca NetBeans'te `System.out` için kaynak koda giderek gösterdi. `println` birçok kez tanımlıdır ve her birinde parametre farklıdır. Bu tanımların bulunduğu sınıf `PrintStream`'dir. Hoca bu sınıfı işletim sistemiyle haberleşen bir sınıf olarak andı. `char` dizisi gibi hemen hemen bütün durumlar yazılmıştır, en sonda da hepsini kapsayan `Object` parametreli sürüm vardır. Aşırı yükleme budur. Javac ile derlenen yansıma denetiminde JDK 25'te `println` metodunun 10 sürümü listelendi: parametresiz, `boolean`, `char`, `int`, `long`, `float`, `double`, `char[]`, `String`, `Object`.

Aynı sınıfta yapıcılar da aşırı yüklenmiştir. IDE'lerde `System.out.println` yazınca çok sayıda seçeneğin çıkmasının sebebi de budur: IDE o sınıfta `println` ile başlayan her şeyi sıralar. Kendi sınıfınızda da aynı şey olur.

20.5 Üçüncü Tür: Operatörler

Hoca sınıfa sordu: Başka neyi aşırı yükleyebiliriz?

Soru: Metot ve yapıcı dışında başka neyi aşırı yükleyebiliriz? **Cevap:** Operatörleri. Ama Java'da operatörler aşırı yüklenemez. C# ve C++'ta bu mümkündür.

Hoca, Java'da olmamasının bunu bilmemeyi gerektirmediğini, tam tersine bilmek gerektiğini söyledi. Bilmeyince farklı bir hata yapılıyor. Onu bir sonraki bölümde görüyoruz. Javac ile denendi: İki `Book` nesnesini `+` ile toplamaya çalışmak `bad operand types for binary operator '+'` hatası veriyor.

21. Nesneleri Karşılaştırmak: == ve equals

21.1 Sık Yapılan Hata

Hoca, öğrencilerin çok yaptığı bir hatayı gösterdi. İki `Book` nesnesi oluşturuluyor. İki alanı var ve değerleri birebir aynı. Sonra aşağıda bu kitapların eşit olup olmadığına bakmak için `==` kullanılıyor.

```
class Book {
    private int id;
    private String name;

    public Book(int id, String name) {
        this.id = id;
```

```
        this.name = name;
    }
}

public class Test {
    public static void main(String[] args) {
        Book b = new Book(1, "Java");
        Book b1 = new Book(1, "Java");
        if (b == b1) {
            System.out.println("esit");
        } else {
            System.out.println("esit degil");
        }
    }
}
```

Soru: Bu `if` `true` mu olur, `false` mu? **Cevap:** `false` olur, çıktı `esit degil`'dir (javac ile denendi).

Sebebi başa dönmektir. `new` ile oluşan nesne heap'te durur, stack'teki değişken ise onun adresini saklar. Bunlar farklı nesneler olduğu için referansları farklıdır. Bir öğrenci de "`b` ve `b1`'i işaretçi gibi düşünebilir miyiz?" diye sordu. Hocanın cevabı bu mantığı doğruladı: Değişkenler bellekte farklı adreslerde oturan nesnelerin referanslarını tutar. İçerik yüzde yüz aynı olsa bile sonuç hep `false` olur.

> **Hocanın vurgusu:** Nesneleri == ile karşılaştırmayın. Bunu yaparsanız sonuç hep `false` gelir ve hata yaparsınız. Hoca, "hocam sonuç yanlış geliyor" denmesinin en büyük sebebinin bu olduğunu söyledi.
>
> **Ders sonrası not:** Hoca "hiç istisnası yok" dedi. Bu söz, ayrı ayrı `new` ile oluşturulan nesneler için geçerlidir. Javac ile denendi: `Book b2 = b;` yazılınca iki değişken aynı nesneyi gösteriyor ve `b == b2` sonucu `true` çıkıyor.

Bu sırada sohbette `Integer` ve `String` için =='ın çalıştığı söylendi. Hoca bunların sarmalayıcı (wrapper) sınıflar olduğunu ve ayrıca bakılacağını söyledi.

> **Ders sonrası not:** Javac ile denendi: `"Java" == "Java"` ve `Integer` için `100 == 100` karşılaştırmaları `true`, `new String("Java") == new String("Java")` ve `Integer` için `1000 == 1000` karşılaştırmaları `false` veriyor. `equals` ise bu iki `false` durumunda `true` veriyor. Yani == burada da içeriği güvenilir biçimde karşılaştırmıyor. `String` ise büyük olasılıkla sarmalayıcı sınıf sayılmaz, çünkü sarmalayıcı sınıflar büyük olasılıkla yalnız ilkel tiplere karşılık gelir.

21.2 equals Metodu

Peki alanları gerçek anlamda nasıl karşılaştıracağız? Bir öğrenci alanlara tek tek erişip karşılaştırmayı önerdi: `b.id` gibi. Hoca bunun için bir fonksiyon yazmak gerektiğini, bu amaçla Java'da hazır bir metot adı bulunduğunu söyledi: `equals`.

```
b.equals(b1);
```

Ama bu metodu kullanmak tek başına yetmez, içeriğini kendi amacımıza göre tanımlamamız gerekir. `equals` kalıtımla gelir, çünkü Java'da bütün nesneler `Object` sınıfından türer. Javac ile denendi: Boş bir `Book` sınıfının üst sınıfı `java.lang.Object` çıkıyor ve hiçbir tanım yapılmadan iki ayrı nesnede `equals` sonucu `false` oluyor.

== operatörü referans tipli verilere böyle tanımlanmış olsaydı karşılaştırmayı onunla yapabilirdik. Bunun için operatörü yeniden tanımlamak gerekirdi, işte buna operatör aşırı yükleme denir. Java'da bu olmadığı için bilmemiz gereken iki sonuç vardır: Java'da operatör aşırı yükleme yoktur ve bu yüzden nesneleri == ile karşılaştırmayacağız. Hoca bunun örneğini sonraki haftada C++ ya da C# ile yapacağını söyledi. `equals` metodunu tanımlamayı da override konusu gelince göreceklerini belirtti.

Sınıfı biz tasarladığımız için istediğimiz metodu yazabiliriz. `equals` olmasaydı "eşit mi" diye kendimiz bir metot yazacaktık. Java, bu çok kullanılan işi "kendi çıkarına göre değiştir, kullan" diye sunmuştur.

Hoca bir öğrencinin sorusu üzerine metodun taslağını sözlü olarak yazdı: `Book` türünde bir `b` nesnesi al, `this.id == b.id` ve `this.name == b.name` şartlarının hepsi sağlanıyorsa `return 1`, en az biri bozuksa `return 0` gönder. Dönüş tipi konusunda da "herhalde boolean değer döndürüyordur, veya integer döndürsün, biz C biliyoruz" dedi.

> **Ders sonrası not:** `Object.equals` metodunun dönüş tipi `boolean`'dır (javac ile derlenen yansıma denetiminde görüldü). `public int equals(Book b)` derleniyor, ama `Object`'teki metodu override etmiyor, ayrı bir aşırı yükleme oluyor. `public int equals(Object b)` ise `equals(Object) in Book cannot override equals(Object) in Object` hatası veriyor. `this.name == b.name` de `String` alanlarda güvenilir değildir: `new String("Java")` ile kurulan iki kitapta sonuç `false` çıkıyor. `equals(Book)` sürümü, `Object` tipli değişkenle çağrılınca içerik aynı olsa bile `false` veriyor. Aşağıdaki sürümde bu sorunlar yoktur (hepsi javac ile denendi).

```
class Book {
    private int id;
    private String name;

    public Book(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Book)) {
            return false;
        }
        Book b = (Book) o;
        return this.id == b.id && this.name.equals(b.name);
    }
}

public class Test {
    public static void main(String[] args) {
        Book b = new Book(1, "Java");
        Book b1 = new Book(1, "Java");
        Object o = b1;
        System.out.println(b == b1);
        System.out.println(b.equals(b1));
        System.out.println(b.equals(o));
    }
}
```

Bu kodun çıktısı sırasıyla `false`, `true`, `true` olur.

22. Akış Kontrolü ve main Metodu

22.1 Akış Kontrolü

Akış kontrolü (control flow), program kodlarının çalışma sırasıdır. Hoca burada `if` gibi deyimleri kastetmediğini belirtti. Yapısal programlamada ilk satırdan başlar, son satıra doğru gideriz. Belki başka bir fonksiyona gider, döner, kaldığımız yerden programın sonuna kadar devam ederiz. Farklı sınıflar yoktur.

Java ve diğer nesne yönelimli dillerde ise program çalışırken nesne oluşturma (`Book b = new Book()`) ya da metot çağrısı (`System.out.print`) ile karşılaşırız. Nesne oluşturmada yapıcıya gidilir, referans döner ve program kaldığı yerden sürer.

Alt düzeyde program; karşılaştırma, toplama, taşıma gibi komutlardan oluşur. Komutların yazılış sırası ile çalışma sırası aynı olmak zorunda değildir. Nesne yönelimli programlarda bu iki sıra çoğunlukla birbirinden oldukça farklıdır.

Sınıf içinde de sıra zorunlu değildir. Değişkenleri en tepeye yazmak şart değildir: Metotlar yukarıda, `x` ve `y` aşağıda olabilir. Yapıcı da aşağıda bir yerde durabilir. Okunabilirlik için önce değişkenler, sonra yapıcı, sonra diğer metotlar yazılır. Yani UML'e uygun yazmak en güzelidir.

22.2 Başlangıç Noktası: Test Sınıfı

Akışın başlangıç noktasını bulmak kolaydır. Projede birçok sınıf ve nesne tanımlayabiliriz, ama projede `main` metodunu içeren bir test sınıfı bulunmalıdır. Buna test ya da sürücü (driver) sınıfı denir. C'de program nasıl `main`'den başlıyorsa, nesne yönelimli programda da `main` metodu bulunan bir sınıf olur.

`main` metodu `static`'tir ve başka türlü olamaz, çünkü program başlarken henüz hiçbir nesne yoktur. `static` olmasaydı, programı çalıştırmak için test sınıfından nesne üretmemiz gerekecekti. Ama program çalışmadan nesne üretilemez. Bu yüzden metot nesne üretmeden sınıf adıyla çağrılır.

22.3 Program Argümanları

`main` metodunun `String[] args` parametresi, programa başlangıç parametreleri aktarmak için kullanılır. Hoca C'de ne eklediğimizi sordu.

Soru: C'de `main` metoduna ne ekliyorduk? **Cevap:** Argüman sayısı (`argc`) ve argüman vektörü (`argv`). Bunlar kaç argüman eklendiğini ve değerlerini söylüyordu.

Java'da sayı için ayrı bir parametre yoktur. Java dizileri referans tipli sakladığı için dizi kendi eleman sayısını da bilir. Bu sayı `args.length` ile alınır. Bu yalnız `String` dizileri için değil, `int` dizileri için de geçerlidir (javac ile denendi).

22.4 main Metodunun Görevi

`main` metodunun görevi ilk nesneleri oluşturmak ve programı başlatmaktır. `main` çalıştıktan sonra istersen bir satır, istersen bin satır yaz, program çalışmaya devam eder. Üretilen nesneler, iç içe nesnelerin içinde üretilen nesneler böyle sürüp gider.

Burada mesaj geçişlerini hatırlayalım. Nesne üretilince yapıcı çağrılır, yapıcıdan geri dönülür, bir metot çağrılır. Metot çağırmak da bir mesaj geçişi gibidir.

Bir sınıfta `main` metodunun bulunması, o metodun illa kullanılacağı anlamına gelmez. Genelde test sınıfındaki `main` çalıştırılır ve program oradan yürür.

22.5 Blok ve Gövde

Süslü parantezlerle yazılan blok ve gövde, birden çok komut içeren bir kod parçasını belirtir. `if`, döngüler ve metotlarda kullanılır. Hoca istenirse adsız (anonymous) bir amaçla da kullanılabileceğini, bunun sonra konuşulacağını söyledi.

23. Car Örneği: Kendi main Metodu Olan Sınıf

23.1 Kod

Dersin son örneği `Car` sınıfıdır. Sınıfta `private` bir `plate` değişkeni vardır. Dönüş tipi olmayan `Car(String plateNr)` yapıcıdır ve `plate` alanına ilk değerini verir. `getPlate` `String` döndüren `public` bir metottur. `plate`, sınıfın bütün metotlarının doğrudan erişebildiği bir değişkendir, önünde bir engel yoktur. `setPlate` bu alana değer atar.

```
package nyp01a;

public class Car {
    private String plate;

    public Car(String plateNr) {
        plate = plateNr;
    }
```

```
    public String getPlate() {
        return plate;
    }

    public void setPlate(String plate) {
        this.plate = plate;
    }

    public void introduceSelf() {
        System.out.println("My plate: " + getPlate());
    }

    public static void main(String[] args) {
        Car aCar;
        aCar = new Car("34 RA 440");
        aCar.introduceSelf();
    }
}
```

Bu kodun çıktısı `My plate: 34 RA 440` olur (javac ile denendi).

`introduceSelf` nesne yönelimli programlamada genel amaçlı ve çok kullanılan bir terimdir: Nesnenin plaka, ad gibi bütün bilgilerini yazar. Bu metotta `getPlate()` yerine doğrudan `plate` de yazılabilirdi. Modüler yaklaşım yüzünden `getPlate` üzerinden dolaylı erişim tercih edilir. Hoca, bu dolaylı çağrının belki biraz süre maliyeti olabileceğini, ama okunabilirlik ve anlaşılırlık için böyle yazıldığını söyledi.

23.2 Bir Sınıf Kendinden Nesne Üretebilir mi?

`main` içinde `Car` sınıfının kendisinden nesne üretiliyor. Bu örnekte sınıf hem tanımlanıyor hem de test ediliyor. Bu mümkün müdür? Tabii ki mümkündür. `new Car` denince bellekte yer açılır. `Car` sınıfının tek bir özelliği olduğu için nesne yalnız bir `String` tutar. Yapıcı bu `plate` özelliğine değeri atar, değişken de referansı tutar.

> **Hocanın vurgusu:** `main` içinde kendi sınıfından nesne üretmek özyineleme (recursion) değildir. Hoca bunun ilk başlarda çok yapılan bir yanlış anlama olduğunu söyledi. `main`'e kadarki metotlar sınıfın normal metotlarıdır. `main` ise bu sınıfı kendi içinden yürütmeyi sağlar. Çalışınca bir nesne üretilir, değerler atanır, `introduceSelf` çağrılır. Özyinelemeli bir durum yoktur.

23.3 Etkileşim Akışı

Slayttaki etkileşim diyagramı şu adımları gösterir:

1. `main`, `Car` sınıfı içinde çalışır ve `new` ile bir `Car` nesnesi oluşturur.
2. Yapıcı çalışır, nesnenin referansı döner ve `aCar` değişkenine yazılır.
3. `main`, nesnenin `introduceSelf` metodunu çağırır.
4. `introduceSelf` kendi içinden `getPlate` metodunu çağırır.
5. `getPlate` plakayı döndürür.
6. `introduceSelf` plaka bilgisini ekrana yazar, çıktı kullanıcıya görünür.

23.4 private Alana Aynı Sınıftan Erişim

Hoca bir öğrencinin sorusuna cevap verdi. `main` aynı sınıfın içinde olduğu için doğrudan `aCar.plate` de yazılabilir. Ama başka bir sınıf olsaydı bu alana doğrudan erişilemezdi. Ya yapıcıyla ya da `set` ve `get` metotlarıyla erişilmesi gerekirdi. Javac ile denendi: Başka sınıftan `c.plate` yazılınca `plate has private access in Car` hatası alınıyor.

23.5 Problemden Koda

Bir problem verildiğinde önce sınıf şeması çıkarılır, sonra bu şema kodlanır. UML üreteçlerinde diyagramı çizip kod üret dediğinizde metotların içi boş bir iskelet elde edersiniz. Tersine mühendislik (reverse engineering) araçlarında ise kodu verip diyagramı elde edersiniz.

Hangi metodun hangisini çağıracağını, hangi değerlerin nasıl geleceğini ise etkileşim diyagramı gösterir. Metotların içini doldurmak yapısal programlama gibi bir algoritma işidir. `introduceSelf` için algoritma, plakayı ya da diğer bilgileri yazdırmaktır.

24. Car Örneğinin İkinci Sürümü ve Yanlış Tasarım

24.1 İkinci Sürüm

Hoca, UML üzerinden kodlamanın özellikle çok kişili projelerde gerekli olduğunu söyledi. Değişken adlarının tutarlı olması gerekir ve herkes her sınıfı kodlamaz. Size bir UML verilir, siz de ondan kodlarsınız.

İkinci sürümde iki `private` değişken ve iki parametre alan tek bir yapıcı vardır. `getPlate`, `setPlate` ve şasi numarası için yalnız bir getter bulunur. Şasi numarası sonradan değiştirilemez, bu yüzden setter'ı yoktur. `main` metodu da yoktur.

```
+-------------------------------------------+
| Car                                       |
+-------------------------------------------+
| - plate : String                          |
| - chassisNR : String                      |
+-------------------------------------------+
| + Car(plateNr : String, chassisNR : String) |
| + getPlate() : String                     |
| + setPlate(plate : String)                |
| + getChassisNR() : String                 |
+-------------------------------------------+
```

```java
public class Car {
    private String plate;
    private String chassisNR;

    public Car(String plateNr, String chassisNR) {
        plate = plateNr;
        this.chassisNR = chassisNR;
    }

    public String getPlate() {
        return plate;
    }

    public void setPlate(String plate) {
        this.plate = plate;
    }

    public String getChassisNR() {
        return chassisNR;
    }
}
```

`main` olmadığı için bu sınıfı tek başına çalıştırıp test edemeyiz. Ayrı bir test sınıfı yazılır. Hoca bu sınıfı sözlü tarif etti: `public class Test` içinde `main` yazılır, `Car c = new Car` ile nesne oluşturulur, sonra `getPlate`, `setPlate` ve şasi numarası metotları denenir. Aşağıdaki argümanlar yer tutucudur (ara adım eklendi).

```java
public class Test {
    public static void main(String[] args) {
```

```
        Car c = new Car("plaka", "sasi");
        System.out.println(c.getPlate());
        c.setPlate("yeni plaka");
        System.out.println(c.getPlate());
        System.out.println(c.getChassisNR());
    }
}
```

Çıktı sırasıyla `plaka`, `yeni plaka`, `sasi` olur (javac ile denendi). Şasi numarasını sonradan değiştirmeye çalışmak ise derlenmez: Sınıfta `setChassisNR` olmadığı için `cannot find symbol` hatası alınır (javac ile denendi).

24.2 Neden Tek Yapıcı Doğru Tasarım?

Gerçek dünyada her arabanın plakası ve şasi numarası vardır. Bu yüzden iki alan da iki parametre alan aynı yapıcıda ilk değerini almalıdır. Araba üretildiği anda bu değerler atanır. Slayttaki iki koddan soldaki doğru, sağdaki yanlıştır.

Sağdaki kodda aynı imzaya sahip iki yapıcı yazılmıştır. Plaka için `Car(String)`, şasi numarası için yine `Car(String)` tanımlıdır. İki imza aynı olduğu için aşırı yükleme burada çalışmaz. Slayt bu kodun derlenmediğini, derlenseydi de mantığının yanlış olacağını belirtir. Javac ile denendi: `constructor Car(String) is already defined in class Car` hatası alınıyor.

> **Ders sonrası not:** Hoca bu durum için önce "execution error", hemen sonra "compile time error" dedi. Doğrusu derleme hatasıdır (javac ile denendi).
>
> **Hocanın vurgusu:** Nesne üretildikten sonra değişmemesini istediğiniz bir alan varsa, önce onu `private` yapın, sonra açık kapı bırakmayın. Şasi numarası için `get` yazılabilir, ama `set` yazılmaz, çünkü ileride değiştirilebilir. Tek kapı yapıcıdır ve yapıcı nesne başına bir kez çağrılır.

Hoca bir de varsayım yaptı: "Velev ki" yapıcılardan biri silinmiş olsun ve tek bir parametre alınsın. Bu kod derlenir, ama tasarım açısından yine yanlıştır. Şu anki hâliyle ise hem derlenmez hem de tasarım açısından yanlıştır.

Soru: Derlenseydi şasi numarası `null` mı olacaktı? **Cevap:** Evet. Şasi numarası `String` olduğu için, yani referans tipli olduğu için `null` olur (javac ile denendi: `getChassisNR()` sonucu `null`).

> **Ders sonrası not:** Hoca cevabın devamında "new ile üretilen her şey null tipindedir" dedi. Doğrusu, değer atanmamış referans tipli alanların `null` başlamasıdır. `new` ile üretilen nesnenin kendisi `null` değildir. Javac ile denendi: `f(new Book())` içinde `b == null` sonucu `false` çıkıyor.

25. this Anahtar Kelimesi

Dersin sonunda bir öğrenci slayta dönüp sordu.

Soru: `public Car` içinde `plate` ve şasi numarasını tanımlarken neden `this.chassisNR` yazıyoruz? **Cevap:** Hoca bunun iki yönünü anlattı.

Birincisi ad karışıklığını önlemektir. Sınıftaki değişkenin adı `plate`, metoda gelen parametrenin adı da `plate` olabilir, buna engel yoktur. Ama o zaman `plate = plate` yazmak gerekir ve ikisi birbirine karışır. `this.plate = plate` yazınca sol taraf sınıfın alanını gösterir. Javac ile denendi: `plate = plate;` yazılan setter alanı değiştirmiyor, alan `null` kalıyor. `this.plate = plate;` yazılınca alan değişiyor.

İkincisi, `this` o anda üzerinde çalışılan nesneyi gösterir. Örneğin `Car` sınıfından `c` adlı bir nesne oluşturuyorum ve `c.getPlate()` çağırıyorum. Bu nesnenin bellekte `plate` ve şasi numarası vardır. `c`, benim dışarıda verdiğim bir değişken adıdır, sınıfın içinde `c` diye bir şey yoktur. İçeride `c`'nin yerine `this` kullanılır. Yani "bu nesneye ait `plate`'i getir" denmiş olur. Hoca slaytta bir atamada `this` kullanıldığını, öbüründe kullanılmadığını da gösterdi. Adlar farklı olunca karışıklık yoktur.

Ders sonrası not: Hoca `this` için “globaldeki değişkene erişmeyi sağlar” dedi. Burada kastedilen, büyük olasılıkla metodun yerel kapsamı dışındaki sınıf alanıdır. Yukarıdaki denemede görüldüğü gibi `this.plate`, o anda çalışılan nesnenin alanını gösterir.

Dersin Toparlaması

Bu derste nesne somut, sınıf soyut bir şablon olarak tanımlandı. Kapsülleme ile veri ve metotlar bir araya getirildi, bilgi gizleme ile alanlar `private`, erişim metotları `public` yapıldı. UML'de sınıf ve nesnenin nasıl yazıldığı, görünürlük işaretleri ve mesaj gösterimi görüldü. `new` ile nesne oluşturmanın iki operatörlü sırası, heap ve stack ayrımı, `static` ile ortak üye paylaşımı, `final` ile tek seferlik atama, yapıcılar, varsayılan değerler ve aşırı yükleme işlendi. Sonunda == ile `equals` farkı, `main` metodunun neden `static` olduğu ve `this` üzerinden `Car` örneği bağlandı.

Hoca, bu konuları kapsayan daha geniş bir örneği doğrudan Java'da kodlayarak göstereceğini söyledi. Belki biraz UML de çizilecek. Ayrıca operatör aşırı yükleme örneği C++ ya da C# ile gösterilecek, `equals` metodunun nasıl tanımlanacağı ise kalıtım ve override konusuyla birlikte ele alınacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.